Live Rubik's Cube Solver: Benchmark of Open-Source Options

I tested the open-source options for every part of a live, camera-guided cube solver on the same tests, to decide what to build on. This report covers everything I ran, the numbers, and what I'm choosing.

Summary: the winners

SOLVER

kociemba

20.7 moves avg, 11 ms (current app: 110)

COLOUR NAMING

Rebalance + resolver

98% of cubes fully right (normal light)

FINDING THE CUBE

YOLO pose (trained)

100% found, 5.2% point error, 15 ms

LIVE MOVE TRACKING

YOLO + 7-point method

98.7% correct, 0.0% wrong (normal light)

Normal light = even lighting and few fingers in the way. Hard conditions = strong colour casts, glare, more fingers, blur and a noisy camera.

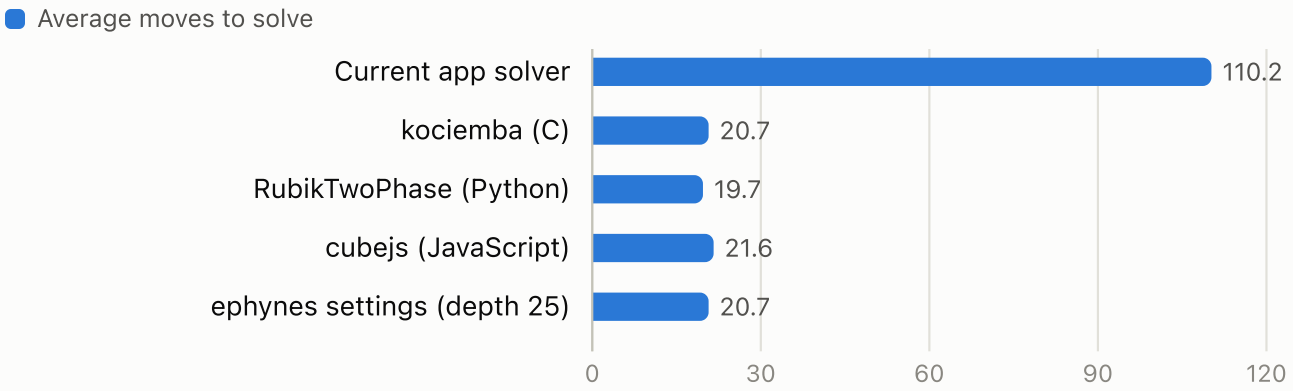
> **No single open-source project does the whole job.** None of them checks the user's moves live. The pipeline I'm building combines the best piece for each step and adds the move tracking myself.

How I tested

Test images	A renderer I wrote draws a 3D cube in any scrambled state (states come from our own cube model) on cluttered backgrounds, with random lighting, colour casts, glare, fingers, blur, camera noise and JPEG compression. Every sticker position is known exactly, so every result can be scored automatically.
Real photos	dwalton76's published photos of one real cube (6 faces, held by a robot) as a real-world check.
Fairness	Each project's own code ran with its own settings. Where a fixed size limit made a project fail on our frame sizes (qbr, ephynes), I also ran a **multi-scale variant** that retries on scaled copies of the frame, labelled "our tweak". The C# project could not run here (no .NET/Unity), so I ported its detection and colour code to Python line by line with all original thresholds.
Scoring	Sticker finding counts only if all 9 stickers land on the right stickers in the right order. Colours count whole cubes with all 54 stickers right. Tracking counts a trial correct only if it names exactly what happened.
Machine	Apple M1 Pro laptop.

1 · Solver

The same 200 random cubes through every solver; each solution was replayed on our cube model to confirm it really solves.



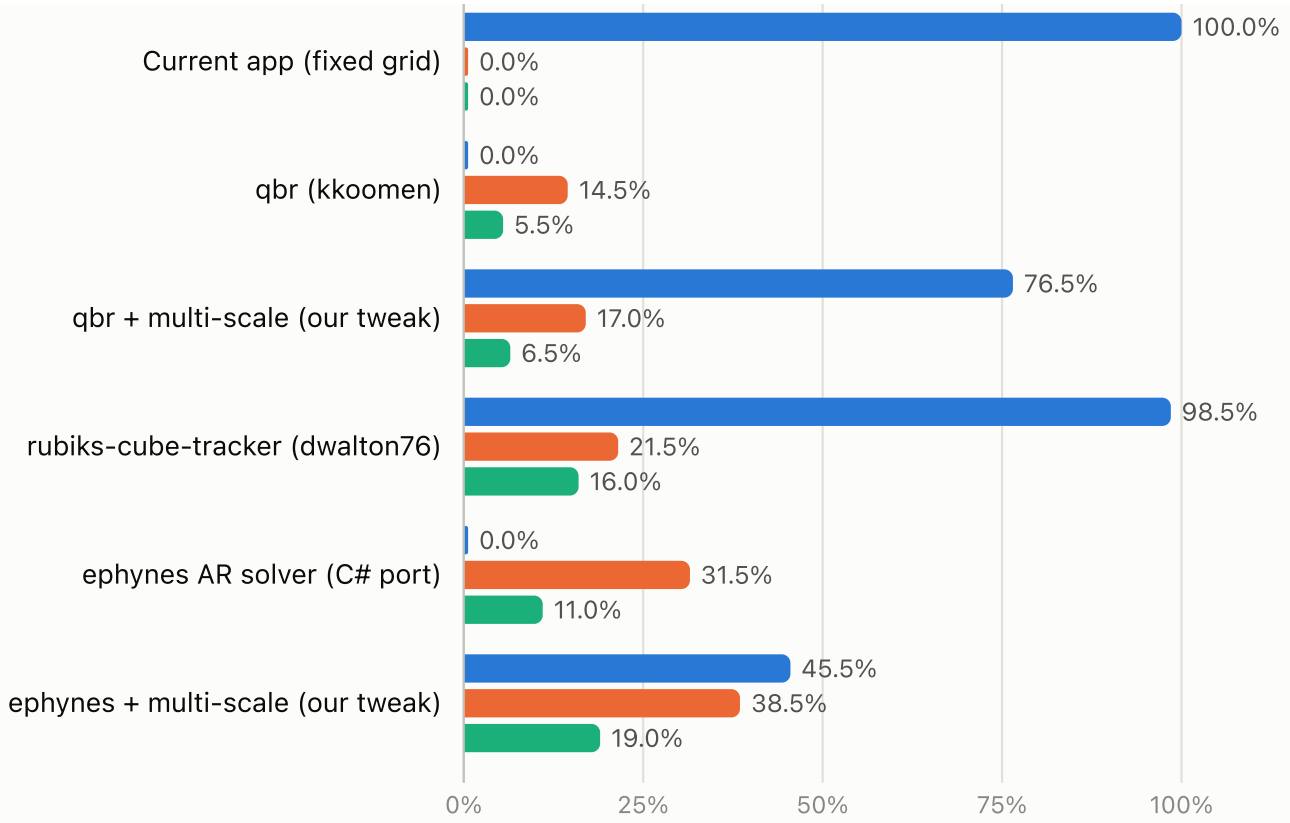
Solver	Solved	Avg moves	Max moves	Median time
Current app solver	100%	110.2	155	51 ms
kociemba (C)	100%	20.7	22	11 ms
RubikTwoPhase (Python)	100%	19.7	20	24 ms
cubejs (JavaScript)	100%	21.6	22	39 ms
ephynes settings (depth 25)	100%	20.7	22	9 ms

kociemba wins. Every solver solves 100% of cubes, but our current solver needs about 5× more moves. kociemba is the fastest of the short-solution solvers, so re-planning after a mistake takes milliseconds.

2 · Finding the stickers during the scan

200 frames per condition. "Right order" means all 9 stickers found in the correct positions.

Aligned to guide grid, normal light · Free hand-held, normal light · Free hand-held, hard conditions

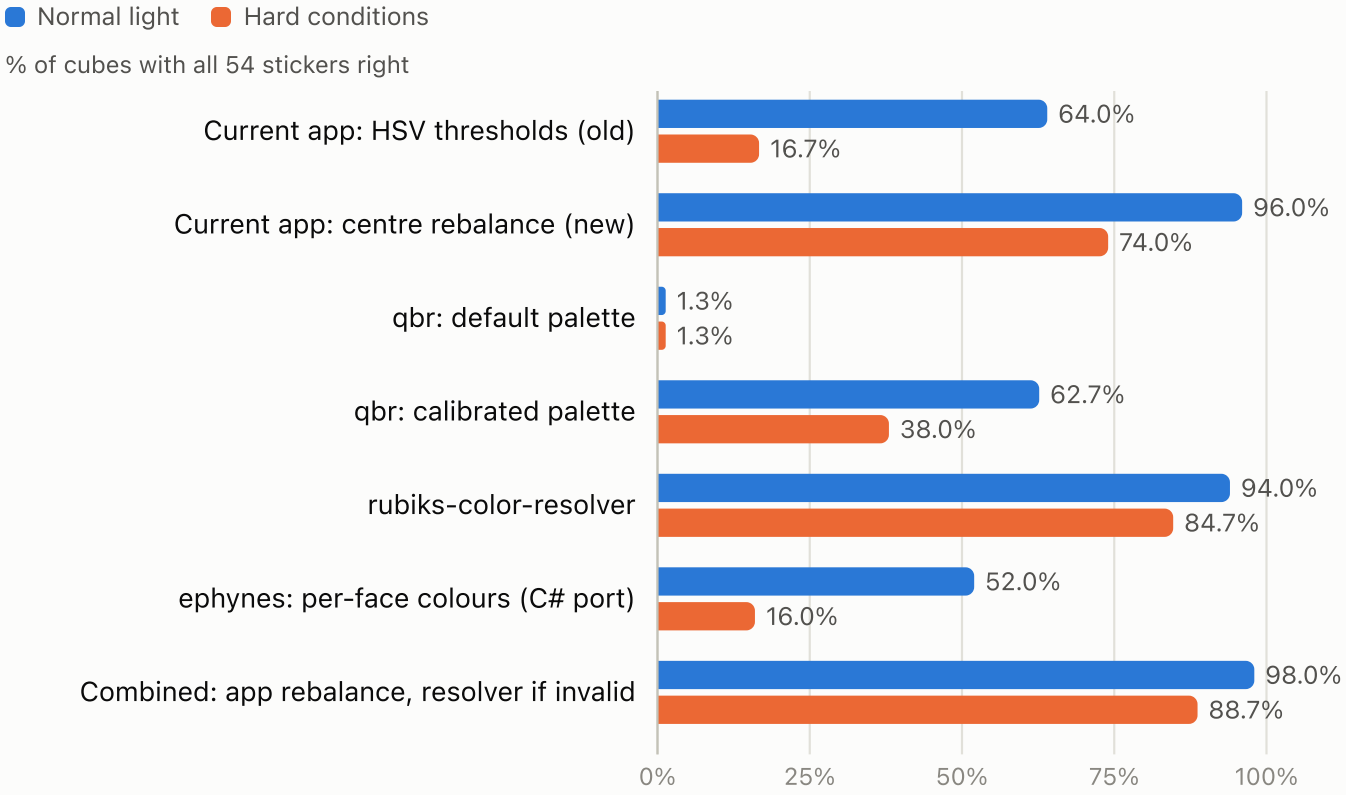


Detector	Free, normal: right order	any rotation	Free, hard: right order	any rotation	Speed	Real photos
Current app (fixed grid)	0.0%	0.0%	0.0%	0.0%	0.0 ms	0/6
qbr (kkoomen)	14.5%	14.5%	5.5%	5.5%	1.0 ms	0/6
qbr + multi-scale (our tweak)	17.0%	17.0%	6.5%	6.5%	3.3 ms	6/6
rubiks-cube-tracker (dwalton76)	21.5%	21.5%	16.0%	16.0%	90.7 ms	6/6
ephynes AR solver (C# port)	31.5%	55.5%	11.0%	26.5%	2.2 ms	0/6
ephynes + multi-scale (our tweak)	38.5%	71.0%	19.0%	44.5%	5.5 ms	2/6

No classical detector is good enough for a hand-held cube. Lined up with a guide grid, our current app and rubiks-cube-tracker are near 100%. Held freely, the best (the ephynes detector with my multi-scale tweak) gets the right order in only 38.5% of frames, and it and rubiks-cube-tracker often lock onto the wrong spot or return the grid rotated.

3 · Naming the colours

150 whole cubes per lighting level (6 scans each, one lighting session per cube). Stickers sampled at their true positions, so only the colour step is tested.

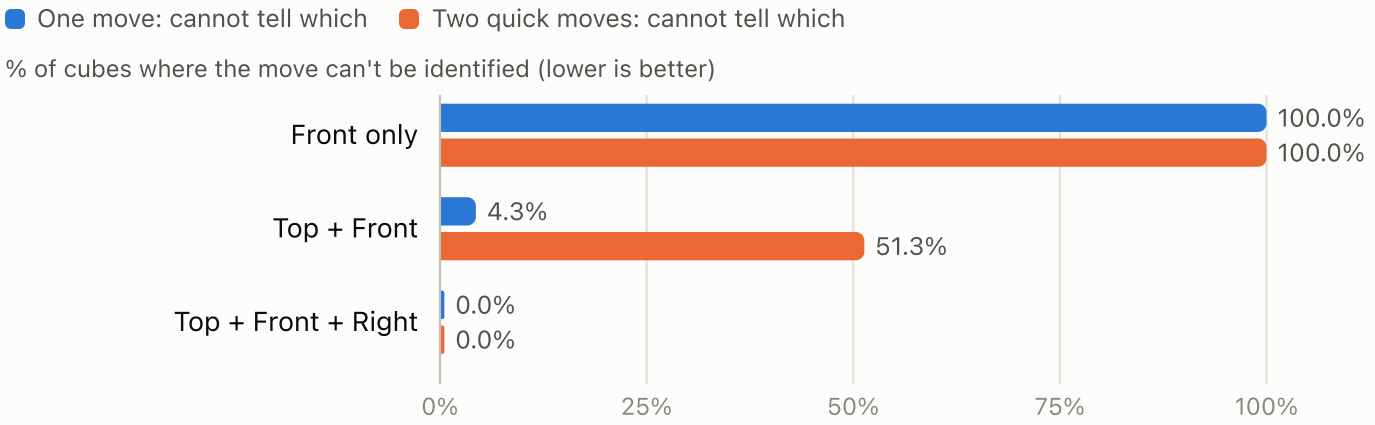


Method	Stickers right (normal)	Stickers right (hard)
Current app: HSV thresholds (old)	99.04%	90.23%
Current app: centre rebalance (new)	99.81%	98.78%
qbr: default palette	74.30%	68.31%
qbr: calibrated palette	99.11%	97.90%
rubiks-color-resolver	99.67%	98.93%
ephynes: per-face colours (C# port)	97.25%	77.99%
Combined: app rebalance, resolver if invalid	99.93%	99.15%

> **Best: our centre-based rebalance, with rubiks-color-resolver as a fallback when the result is invalid**: 98% of cubes fully right in normal light and 88.7% in hard conditions, better than either alone.

4 · How much of the cube the camera must see

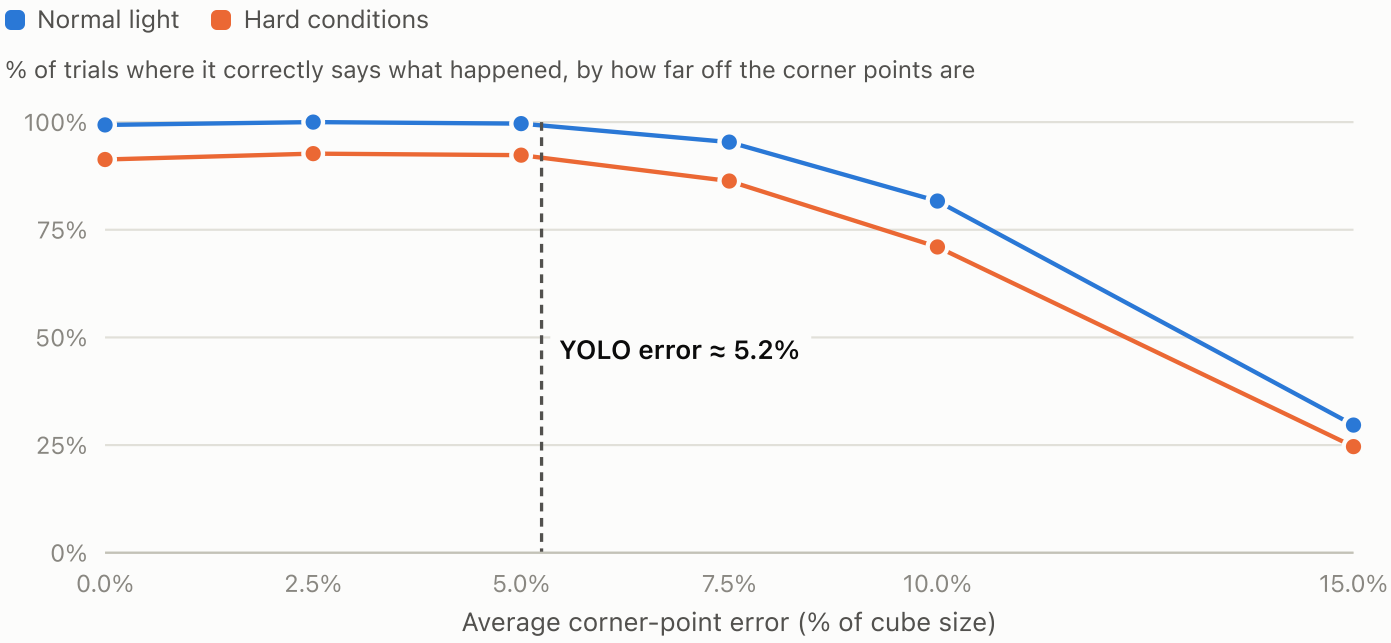
For 300 random cubes I checked, using our cube model, whether every possible move (and every pair of moves) leaves a different picture on the visible sides.



> **The camera must see 3 sides (Top, Front, Right) during the solve.** With the front alone, turns of the back are invisible; with Top + Front, two quick moves are confused half the time; with 3 sides every move and every pair is identifiable.

5 · Live move tracking

300 before/after trials per point: the right move, no move, a wrong move, or two quick moves. The 3 sides are read from the cube's 7 corner points (the sebastienlato/cube-solver method, which I re-implemented), then compared with every possible move.



Average point error	0% (perfect)	2.5%	5.0%	7.5%	10.0%	15.0%
Correct · normal	99.3%	100.0%	99.7%	95.3%	81.7%	29.7%
Wrong · normal	0.0%	0.0%	0.0%	0.0%	0.3%	1.7%
Correct · hard	91.3%	92.7%	92.3%	86.3%	71.0%	24.7%
Wrong · hard	0.0%	0.0%	0.0%	0.3%	0.7%	4.7%

> **The tracking logic works**: with an average corner-point error up to about 5% of the cube size it is 99.7% right in normal light and 92% in hard conditions, and it almost never names a wrong move. When unsure it waits or asks for a rescan instead of guessing.

Can existing detectors handle the corner view?

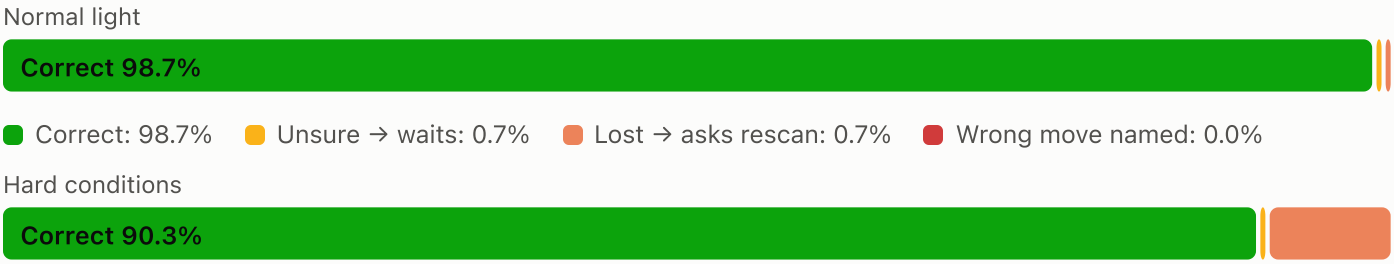
Detector	Found any 1 face (normal)	(hard)	Reads all 3 sides?
qbr (kkoomen)	0.0%	0.0%	No: built for one flat face
qbr + multi-scale (our tweak)	0.0%	0.0%	No: built for one flat face
rubiks-cube-tracker (dwalton76)	2.7%	0.0%	No: built for one flat face
ephynes AR solver (C# port)	4.0%	1.3%	No: built for one flat face
ephynes + multi-scale (our tweak)	4.0%	1.3%	No: built for one flat face
YOLO pose (trained by me)	100.0%	100.0%	Yes: finds the 7 corner points

6 · YOLO to find the cube

I trained a small YOLO pose model (yolo26n-pose) on 2,500 synthetic corner frames (8 passes, 32 min on the M1 Pro) and tested it on 300 new frames per lighting level.

	Normal light	Hard conditions
Cube found	100.0%	100.0%
Median point error	5.2%	5.1%
95% of frames under	7.8%	7.5%
Speed (M1 Pro GPU)	15 ms	16 ms
Speed (CPU only)	40 ms	39 ms
Tracking: correct	98.7%	90.3%
Tracking: wrong move named	0.0%	0.0%
Tracking: unsure / lost (waits or asks rescan)	1.3%	9.7%

Tracking outcomes with YOLO finding the points



Correct: 90.3% Unsure → waits: 0.7% Lost → asks rescan: 9.0% Wrong move named: 0.0%

> **YOLO is what makes live tracking possible.** It found the cube in every test frame, its ~5% point error sits right at what tracking tolerates, and tracking with it never named a wrong move. Every classical detector fails at this view.

7 · The ephynes C# project (AR-Optimal-Rubiks-Cube-solver)

Because our end product runs on XREAL glasses with a C# (Unity) app, I checked this C# AR solver in detail. It is a Unity port of AndroidSteve's "Rubik Cube Wizard", built for a Meta 2 headset study.

Area	ephynes (C#)	Best option found	Verdict
Following moves during the solve	None: advances whenever a stable face reappears; never checks the move	YOLO + matcher: 98.7% correct, 0 wrong	**Missing**
Finding a hand-held face	31.5% right order (38.5% with multi-scale); grid often rotated	YOLO: 100% found	Best classical, not enough
Corner view (3 sides)	4.0%	YOLO: 100%	No
Colours	52% / 16% cubes right	98% / 88.7%	No
Solver	20.7 moves (two-phase, maxDepth 25)	kociemba 20.7	Same
Speed per frame	2.2 ms	YOLO 15 ms (M1 Pro GPU)	Faster

- **No move checking:** during the solve it moves to the next instruction whenever the camera sees a stable face again (`AppStateMachine.cs`), so a wrong move goes unnoticed and nothing is re-planned.
- **Rotated grid:** it orders the 9 stickers from whichever corner of a sticker is highest in the image, so a small tilt left or right can rotate the grid by 90°. Allowing any rotation, it finds the face in 55.5% of frames, versus 31.5% in the right order.
- **Colour check disabled:** its "exactly 9 of each colour" step is commented out and would crash if enabled, so colours are named one face at a time.
- **Worth reusing:** its Unity structure (camera into OpenCvSharp, state machine, Kalman filter) and possibly its C# two-phase solver, once the solver's licence is confirmed (it was translated from a project with no licence file).

> **It does not perform better.** It has the best classical face detector and is very fast, but it can't check moves, which is the core feature I need.

8 · Hardware findings (XREAL One Pro + Beam Pro)

- **Camera:** our glasses have the **XREAL Eye** camera add-on, so the camera sees what the wearer sees, which is the ideal view for this app. XREAL's documentation confirms the Eye is the camera apps can use on the

One series.

- **Compute:** the Beam Pro is an Android 14 device with a Snapdragon 6 Gen 1 and two 50 MP rear cameras. YOLO can run on it in Unity through the Inference Engine (Sentis) from an ONNX export; speed on this mid-range chip still needs measuring.
- **SDK:** the One Pro uses XREAL SDK 3.x (camera class `XREALRGBCameraTexture`, YUV_420_888 frames), not the older NRSDK our Unity scaffold was written against.

What I'm going with

1. **Solver:** kociemba.
2. **Finding the cube:** a YOLO pose model for both the scan and tracking.
3. **Reading 3 sides:** the 7-corner-point method.
4. **Colours:** centre rebalance with rubiks-color-resolver as fallback; re-calibrated from the last frame during tracking.
5. **Move tracking:** compare against every predicted move; wait when unsure, rescan only when lost.
6. **Rollout:** Phase 1 runs on a laptop with its webcam (Python pipeline + browser view). Phase 2, once Phase 1 works, moves it to our XREAL glasses with the Eye camera, with the app in C# on the Beam Pro.

> **Limits.** Almost all images are synthetic, so real-world numbers will be lower; YOLO in particular has only seen synthetic frames and must be fine-tuned on a few hundred real photos of our cubes. The only real photos were 6 photos of one robot-held cube. Tracking was tested on single before/after frames; the real app will also average several steady frames.

Everything I looked at

Project	What I did with it	Licence
muodov/kociemba	Solver benchmark	GPL-2.0
hkociemba/RubiksCube-TwophaseSolver (RubikTwoPhase)	Solver benchmark	GPL-3.0
cubejs	Solver benchmark	MIT
kkoomen/qbr	Sticker finding + colour benchmarks (plus my multi-scale variant)	MIT
dwalton76/rubiks-cube-tracker	Sticker finding + real photos	MIT
dwalton76/rubiks-color-resolver	Colour benchmark	MIT
ephynes/AR-Optimal-Rubiks-Cube-solver (C#)	Read the code; ported detection + colours to Python; all benchmarks	MIT (solver origin unlicensed)
sebastienlato/cube-solver	7-corner-point / homography method, re-implemented (no code copied)	None stated
Ultralytics YOLO (yolo26n-pose)	Trained and tested a cube keypoint model	AGPL-3.0
RubikClear, cube-cv, CubeCV, cubeworks, RubikVision, Bishwaswarup/rubiks, cdbharath/rubiks_cube_3dcv, ios-rubik-solver	Reviewed only: none does live move checking; nothing to add over the tested ones	various